ES-CS301 — Data Structure & Algorithms

Semester III • 3L:0T:4P • 3 Credits

Module 3 Exam Notes: Linked Lists and Tree Data Structures

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1 Linked Lists Fundamentals and Singly Linked List (SLL) . . . 2
1.1 Limitations of Arrays vs Advantages of Linked Lists . . . 2
1.2 Singly Linked List Algorithmic Operations . . . 2
2 Linked Representation of Stack and Queue & Header Nodes . . . 3
2.1 Linked Stack and Linked Queue . . . 3
2.2 Header Nodes in Linked Lists . . . 3
3 Doubly Linked Lists (DLL) and Circular Linked Lists (CLL) . . . 4
3.1 Doubly Linked List (DLL) . . . 4
3.2 Circular Linked Lists (CLL) . . . 4
4 Tree Terminologies and Binary Tree Foundations . . . 4
4.1 Mathematical Properties of Binary Trees . . . 5
4.2 Binary Tree Classifications . . . 5
4.3 Binary Tree Traversals . . . 6
5 Threaded Binary Tree . . . 6
6 Binary Search Tree (BST) . . . 6
6.1 BST Operations and Deletion Cases . . . 7
7 AVL Tree (Self-Balancing Binary Search Tree) . . . 7
7.1 AVL Rebalancing Rotations . . . 7
8 Multi-Way Search Trees: B-Tree and B+ Tree . . . 8
8.1 B-Tree of Order m . . . 8
8.2 B+ Tree and Comparison with B-Tree . . . 8

1. Linked Lists Fundamentals and Singly Linked List (SLL)

Definition: Singly Linked List (SLL) Architecture

A **Singly Linked List** is a dynamic linear data structure composed of distinct nodes allocated at arbitrary non-contiguous physical memory locations. Each node consists of two essential components:

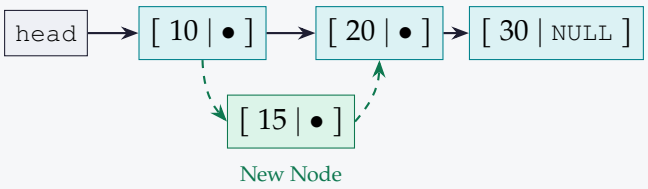
1. **Data Field:** Stores the actual information payload of type T.
2. **Link (Next Pointer):** Stores the explicit physical memory address of the logically succeeding node.

The list is anchored by an external pointer `head` referencing the first node. The terminal node's link contains `NULL`.

1.1 Limitations of Arrays vs Advantages of Linked Lists

- **Memory Allocation:** Arrays require fixed, compile-time contiguous memory allocation. Linked lists allocate memory dynamically on the heap at runtime via `malloc() / free()`, growing and shrinking on demand.
- **Insertion and Deletion Overhead:** Inserting or deleting an element at index 0 in an array of size n requires shifting all n subsequent elements ($\mathcal{O}(n)$ time). In a linked list, inserting or deleting at the head requires updating pointer references, which executes in strictly $\mathcal{O}(\mathbf{1})$ time without moving physical payloads.
- **Memory Fragmentation:** Arrays can suffer allocation failure due to external memory fragmentation even when total free RAM exceeds the array size; linked lists utilize scattered memory chunks effectively.

Memory Representation and Pointer Rewiring in a Singly Linked List



1.2 Singly Linked List Algorithmic Operations

Algorithms for Insertion and Deletion in SLL

1. Insertion at Beginning ($\mathcal{O}(1)$):

```
newNode = allocateNode(val);
newNode->next = head;
head = newNode;
```

2. Insertion at End ($\mathcal{O}(n)$ without tail pointer, $\mathcal{O}(1)$ with tail):

```
newNode = allocateNode(val); newNode->next = NULL;
if (head == NULL) { head = newNode; return; }
```


© Biraj

```
ptr = head;
while (ptr->next != NULL) ptr = ptr->next;
ptr->next = newNode;
```

3. Deletion at Beginning ($\mathcal{O}(1)$):

```
if (head == NULL) report "Underflow";
temp = head;
head = head->next;
free(temp);
```

4. Deletion of a Node with Value X ($\mathcal{O}(n)$):

```
if (head == NULL) return;
if (head->data == X) { temp = head; head = head->next; free(temp); return; }
curr = head; prev = NULL;
while (curr != NULL && curr->data != X) { prev = curr; curr = curr->next; }
if (curr == NULL) report "Key Not Found";
else { prev->next = curr->next; free(curr); }
```

2. Linked Representation of Stack and Queue & Header Nodes

2.1 Linked Stack and Linked Queue

- **Linked Stack:** Implemented by inserting and deleting strictly at the head node of the linked list:
  - Push(x) $\equiv$ Insert at Beginning $\implies \mathcal{O}(\mathbf{1})$
  - Pop() $\equiv$ Delete from Beginning $\implies \mathcal{O}(\mathbf{1})$
  - Overflow never occurs unless the system runs out of heap memory.
- **Linked Queue:** Maintained via two external pointers: `front` and `rear`:
  - Enqueue(x) $\equiv$ Insert at End (via `rear->next` = newNode; `rear` = newNode) $\implies \mathcal{O}(\mathbf{1})$
  - Dequeue() $\equiv$ Delete at Head (via `front` = `front->next`) $\implies \mathcal{O}(\mathbf{1})$

2.2 Header Nodes in Linked Lists

A **Header Node** is a dummy node placed at the very beginning of the linked list before the first actual data node:

- **Grounded Header List:** Header node's link points to the first data node; the last node's link contains `NULL`.
- **Circular Header List:** The last node's link points back to the header node itself.
- **Key Advantages:**
  1. Eliminates special-case coding for operations at the head (head pointer never needs re-assignment).
  2. The list is never truly empty (`head != NULL` always holds; an empty list has `head->next == NULL` or `head->next == head`).

© Biraj

3. The header node's data field can store metadata (e.g., total node count, maximum element).

3. Doubly Linked Lists (DLL) and Circular Linked Lists (CLL)

3.1 Doubly Linked List (DLL)

In a **Doubly Linked List**, each node contains three fields: `prev` pointer, `data`, and `next` pointer.

- **Invariant:** `p->next->prev == p` and `p->prev->next == p`.
- **Bidirectional Traversal:** Allows forward and reverse scanning from any arbitrary node.
- $\mathcal{O}(1)$ **Deletion Given Node Pointer:** Deleting node P does not require searching for its predecessor:

```
P->prev->next = P->next;
if (P->next != NULL) P->next->prev = P->prev;
free(P);
```

- **Trade-off:** Extra memory overhead for backward pointers ($2\times$ pointer overhead per node) and additional pointer adjustments during insertions.

3.2 Circular Linked Lists (CLL)

In a **Circular Singly Linked List**, the last node's link points back to the first node.

- **Tail Pointer Paradigm:** By maintaining a single pointer `tail` to the last node:
  - Last node is accessible in $\mathcal{O}(1)$ (`tail`).
  - First node is accessible in $\mathcal{O}(1)$ (`tail->next`).
  - Both insertion at head and insertion at tail execute in strictly $\mathcal{O}(\mathbf{1})$ time!
- **Applications:** Continuous cycling processes, round-robin CPU scheduling, multiplayer turn games.

4. Tree Terminologies and Binary Tree Foundations

Definition: Tree Data Structure and Core Terminologies

A **Tree** is a non-linear, hierarchical data structure consisting of a collection of nodes connected by directed or undirected edges, such that there exists a unique root node R and no cycles.

- **Root:** The topmost ancestor node with in-degree 0 (no incoming parent edge).
- **Leaf (Terminal Node):** A node with out-degree 0 (no child nodes).

© Biraj

- **Degree of a Node:** The total number of subtrees or immediate children attached to that node.
- **Degree of a Tree:** The maximum degree among all nodes in the tree.
- **Level and Depth:** The root is at Level 0 (or 1). The depth of a node is the length of the path from the root to that node.
- **Height of a Tree (h):** The maximum depth among all nodes in the tree (length of the longest path from root to leaf).

4.1 Mathematical Properties of Binary Trees

A **Binary Tree** is a finite tree in which every node has at most two children, distinguished as **left child** and **right child**.

Core Mathematical Theorems on Binary Trees

1. Maximum number of nodes at level i (with root at level 0):

$$N_{\max}(i) = 2^i \tag{1}$$

2. Maximum total nodes in a binary tree of height h:

$$N_{\max}(h) = \sum_{i=0}^{h} 2^i = 2^{h+1} - 1 \tag{2}$$

3. Minimum height of a binary tree with n nodes:

$$h_{\min} = \lceil \log_2(n+1) \rceil - 1 \tag{3}$$

4. **Leaf vs Degree-2 Nodes Theorem:** In any non-empty binary tree, the number of leaf nodes n_0 is exactly one more than the number of nodes with degree two (n_2):

$$\mathbf{n_0 = n_2 + 1} \tag{4}$$

Proof: Let total nodes be $n = n_0 + n_1 + n_2$. Total edges in a tree of n nodes is $E = n - 1$. Also, each degree-1 node contributes 1 outgoing branch and each degree-2 node contributes 2 outgoing branches: $E = n_1 + 2n_2$. Equating the two expressions:

$$n_0 + n_1 + n_2 - 1 = n_1 + 2n_2 \implies n_0 - 1 = n_2 \implies \mathbf{n_0 = n_2 + 1}$$

4.2 Binary Tree Classifications

- **Full (Strict / Proper) Binary Tree:** Every internal node has exactly two children (0 or 2 children; no node has degree 1).
- **Complete Binary Tree:** All levels are completely filled except possibly the last level, where nodes are filled strictly from left to right without gaps. Foundation of array-based Binary Heaps.
- **Perfect Binary Tree:** All internal nodes have two children and all leaves reside at the same

© Biraj

maximum depth (contains exactly $2^{h+1} - 1$ nodes).

- **Degenerate (Skewed) Binary Tree:** Every internal node has only one child, degrading to a linear linked list ($\mathcal{O}(n)$ height).

4.3 Binary Tree Traversals

1. **Preorder Traversal (D-L-R):** Process Data → Traverse Left subtree → Traverse Right subtree.
2. **Inorder Traversal (L-D-R):** Traverse Left subtree → Process Data → Traverse Right subtree. Produces sorted sequence for a BST!
3. **Postorder Traversal (L-R-D):** Traverse Left subtree → Traverse Right subtree → Process Data. Used for deleting tree bottom-up and expression evaluation.
4. **Level-Order Traversal (BFS):** Visit nodes level by level from top to bottom, left to right, implemented using a FIFO Queue.

5. Threaded Binary Tree

Definition: Threaded Binary Tree

In an n-node binary tree, exactly $2n$ pointer fields exist, of which $n-1$ point to children and $n+1$ are `NULL`. A **Threaded Binary Tree** utilizes these wasted null pointers:
- A `NULL` left pointer is converted into a **thread** pointing to the node's **Inorder Predecessor**.
- A `NULL` right pointer is converted into a **thread** pointing to the node's **Inorder Successor**.

Boolean tags (`lTag`, `rTag`) indicate whether the pointer field is a normal child link (0) or a thread (1).

Primary Advantage: Enables complete Inorder traversal without requiring recursive calls or an auxiliary stack, utilizing $\mathcal{O}(\mathbf{1})$ auxiliary memory space.

6. Binary Search Tree (BST)

Definition: Binary Search Tree (BST) Invariant

A **Binary Search Tree (BST)** is a binary tree in which every node x satisfies the BST ordering property:
- For all nodes y in the left subtree of x: $\text{Key}(y) < \text{Key}(x)$.
- For all nodes z in the right subtree of x: $\text{Key}(z) > \text{Key}(x)$.

Duplicate keys are either disallowed or handled uniformly in right subtrees.

© Biraj

6.1 BST Operations and Deletion Cases

- **Search** ($O(h)$)**:** Compare key with current node: if equal, return success; if less, recurse left; if greater, recurse right.
- **Insertion** ($O(h)$)**:** Search for insertion locus until reaching a null link, then attach new node.
- **Deletion** ($O(h)$) **— Three Cases:**
  1. **Case 1: Node is a Leaf:** Discard node directly and set parent's pointer to `NULL`.
  2. **Case 2: Node has One Child:** Bypass the node by linking the parent directly to the single child.
  3. **Case 3: Node has Two Children:** Replace the node's value with its **Inorder Predecessor** (maximum key in left subtree) or **Inorder Successor** (minimum key in right subtree), then recursively delete that predecessor/successor node (which is guaranteed to have at most one child).

7. AVL Tree (Self-Balancing Binary Search Tree)

Definition: AVL Tree and Balance Factor

An **AVL Tree** is a strictly height-balanced Binary Search Tree where the **Balance Factor** (BF) of every node v is constrained to $\{-1, 0, +1\}$:

$$BF(v) = h(\text{left subtree}) - h(\text{right subtree}) \in \{-1, 0, +1\} \tag{5}$$

If an insertion or deletion causes $|BF(v)| \geq 2$, the tree is rebalanced locally via rotations.

7.1 AVL Rebalancing Rotations

1. **LL Rotation (Single Right Rotation):** Applied when insertion occurs in the left subtree of the left child ($BF = +2$, child $BF = +1$).
2. **RR Rotation (Single Left Rotation):** Applied when insertion occurs in the right subtree of the right child ($BF = -2$, child $BF = -1$).
3. **LR Rotation (Double Rotation):** Insertion in the right subtree of the left child ($BF = +2$, child $BF = -1$). First Left-rotate left child, then Right-rotate parent.
4. **RL Rotation (Double Rotation):** Insertion in the left subtree of the right child ($BF = -2$, child $BF = +1$). First Right-rotate right child, then Left-rotate parent.

Height Bound and Complexity of AVL Trees

The minimum number of nodes $N(h)$ in an AVL tree of height h satisfies the Fibonacci recurrence:

$$N(h) = N(h-1) + N(h-2) + 1, \quad N(0) = 1,\ N(1) = 2 \tag{6}$$

Solving this relation yields an asymptotic upper bound on height:

$$h < 1.44 \log_2(n+2) \implies \mathbf{h = \mathcal{O}(\log_2 n)} \tag{7}$$

© Biraj

Consequently, Search, Insert, and Delete operations in an AVL tree are **guaranteed strictly** $\mathcal{O}(\log \mathbf{n})$ in both average and worst cases!

8. Multi-Way Search Trees: B-Tree and B+ Tree

8.1 B-Tree of Order m

Designed for secondary storage (disk systems) to minimize slow disk I/O operations by maximizing node branching factor.

Axiomatic Properties of B-Tree of Order m

1. Every node has at most m children and $m-1$ keys.
2. The root node has at least 2 children (unless it is a leaf).
3. Every internal node (except root) has at least $\lceil m/2 \rceil$ children and $\lceil m/2 \rceil - 1$ keys.
4. All leaf nodes reside at the exact same depth/level.
5. A non-leaf node with k children contains $k-1$ keys acting as separation boundaries.

8.2 B+ Tree and Comparison with B-Tree

- **B+ Tree Structure:** In a B+ tree, all data records and actual keys are stored **strictly in leaf nodes**. Internal nodes store only search routing keys.
- **Linked Leaves:** All leaf nodes are linked sequentially via pointers to form a doubly linked list, enabling extremely fast $\mathcal{O}(k)$ range queries.
- **Database Indexing Standard:** B+ Trees are universally used in relational database engines (MySQL, PostgreSQL, Oracle) and file systems (NTFS, ext4).

© Biraj

Quick Revision — Module 3

Linked Lists and Tree Data Structures | ES-CS301

Key Formulas, Tree Bounds and Complexity at a Glance

Binary Tree Max Nodes Level i	$N_{\max}(i) = 2^i$
Binary Tree Max Total Nodes	$N_{\max}(h) = 2^{h+1} - 1$
Binary Tree Min Height	$h_{\min} = \lceil \log_2(n+1) \rceil - 1$
Leaf Nodes vs Degree-2 Nodes	$n_0 = n_2 + 1$ (Universal Binary Tree Invariant)
AVL Balance Factor	$BF(v) = h_L(v) - h_R(v) \in \{-1, 0, +1\}$
AVL Maximum Height	$h < 1.44 \log_2 n \implies \mathcal{O}(\log n)$
SLL Head Insert / Delete	$\mathcal{O}(1)$ time, $\mathcal{O}(1)$ auxiliary space
BST Search / Insert / Delete	Average: $\mathcal{O}(\log n)$, Worst (skewed): $\mathcal{O}(n)$
AVL Search / Insert / Delete	Guaranteed $\mathcal{O}(\log n)$ worst-case
B-Tree Min Keys per Internal Node	$\lceil m/2 \rceil - 1$

One-Line Definitions

- **Singly Linked List:** A linear data structure where each node stores a data value and a pointer to the next node.
- **Doubly Linked List:** A linked list where nodes store pointers to both their forward and backward adjacent nodes.
- **Circular Linked List:** A linked list whose terminal node points back to the first node instead of containing `NULL`.
- **Header Node:** A dummy lead node placed before actual data elements to simplify boundary operations.
- **Full Binary Tree:** A binary tree where every internal node has strictly two children.
- **Complete Binary Tree:** A binary tree completely filled at all levels except possibly the last, which is filled left-to-right.
- **Threaded Binary Tree:** A binary tree that replaces null leaf pointers with threads to inorder predecessors and successors.
- **Binary Search Tree:** A binary tree where all left descendants are smaller and all right descendants are greater than the node.
- **AVL Tree:** A self-balancing binary search tree maintaining balance factors of all nodes within $\{-1, 0, +1\}$.

© Biraj

- **B-Tree:** A self-balancing multi-way search tree of order m optimized for block access in secondary storage.
- **B+ Tree:** A B-tree variant where all records reside in linked leaves and internal nodes store only index keys.

Frequently Asked / Viva Questions

Q1. State and prove the relationship between leaf nodes and degree-2 nodes in any non-empty binary tree.

In any binary tree, $n_0 = n_2 + 1$. Proof: Let total nodes be $n = n_0 + n_1 + n_2$. Total edges in a tree of n nodes is $E = n - 1$. Also, each degree-1 node contributes 1 outgoing branch and each degree-2 node contributes 2: $E = n_1 + 2n_2$. Setting $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2$ simplifies directly to $n_0 = n_2 + 1$.

Q2. Why does Inorder traversal of a Binary Search Tree produce sorted output?

Inorder traversal follows the Left-Data-Right (L-D-R) sequence. Because the BST invariant guarantees that all left subtree nodes are smaller than the root, and the root is smaller than all right subtree nodes, recursively visiting Left, then Root, then Right naturally produces a monotonically ascending sequence.

Q3. Explain the four rebalancing rotations of an AVL tree.

Single Right (LL) fixes left-heavy imbalance caused by insertion in left subtree of left child. Single Left (RR) fixes right-heavy imbalance. Left-Right (LR) double rotation fixes left-heavy node with right-heavy child via left rotation on child followed by right on parent. Right-Left (RL) performs right rotation on child followed by left on parent.

Q4. How does a B+ Tree differ fundamentally from a B-Tree?

In a B-tree, keys and associated data records are stored in both internal and leaf nodes. In a B+ tree, all data records are stored strictly in leaf nodes; internal nodes store only routing index keys. Furthermore, leaf nodes in a B+ tree are linked sequentially into a linked list for rapid range scanning.

Q5. What are the advantages of maintaining a Tail pointer in a Circular Linked List?

With only a tail pointer, the last node is directly accessed in $\mathcal{O}(1)$ via `tail`, and the first node is accessed in $\mathcal{O}(1)$ via `tail->next`. This allows insertion at the beginning and insertion at the end to both execute in strictly $\mathcal{O}(1)$ time without traversing the list.

Q6. What is the primary motivation for using Threaded Binary Trees?

In a binary tree of n nodes, $n + 1$ pointer fields are wasted as `NULL`. Threaded trees replace these null pointers with threads pointing to inorder predecessors and successors, permitting full inorder traversal without recursion or an auxiliary stack ($\mathcal{O}(1)$ extra space).

Q7. What is the worst-case time complexity of searching in a BST, and when does it occur?

© Biraj

Frequently Asked / Viva Questions (contd.)

The worst-case search complexity is $\mathcal{O}(n)$. This occurs when keys are inserted in strictly sorted or reverse-sorted order, causing the BST to degenerate into a skewed linear chain where height $h = n - 1$.

Q8. Why are B-Trees preferred over Binary Search Trees for database indexes on disk?

Disk access is significantly slower than RAM access. A BST has high height ($\mathcal{O}(\log_2 n)$), requiring many sequential disk seeks. A B-Tree has a high branching factor m (order $m \approx 100$ to 1000), keeping height very low ($\mathcal{O}(\log_m n)$), so only 2 to 3 disk I/O reads are needed to retrieve any record.

Q9. Explain the three cases of node deletion in a Binary Search Tree.

Case 1: Leaf node (simply remove and nullify parent's pointer). Case 2: Node has one child (bypass node by connecting parent directly to child). Case 3: Node has two children (replace value with inorder successor or predecessor, then delete that node which has at most one child).

Q10. What is the purpose of a Header Node in a linked list?

A header node simplifies boundary conditions: it ensures the list is never empty (head pointer never changes) and eliminates special-case logic for inserting or deleting at the front of the list.

Summary Matrix: Comprehensive Comparison of Tree Data Structures

Tree Structure	Structural Invariant	Search Complexity	Primary Application Domain
Binary Tree	**At most 2 children per node**	$\mathcal{O}(n)$	Expression trees, hierarchical parsing, Huffman coding.
Threaded Binary Tree	**Null pointers store inorder links**	**Traversal:** $\mathcal{O}(1)$ **space**	Stackless tree traversal in memory-constrained systems.
Binary Search Tree (BST)	**Left** $<$ **Root** $<$ **Right**	**Avg:** $\mathcal{O}(\log n)$, **Worst:** $\mathcal{O}(n)$	Dynamic searching, symbol tables, dictionary ADT.
AVL Tree	**Strict balance:** $\lvert BF \rvert \leq 1$	**Guaranteed** $\mathcal{O}(\log n)$	Search-intensive in-memory datasets and lookup tables.
B-Tree	**Multi-way:** $\lceil m/2 \rceil \leq$ **children** $\leq m$	$\mathcal{O}(\log_m n)$	File systems, relational databases, storage indexing.
B+ Tree	**Data in leaves; leaves linked**	$\mathcal{O}(\log_m n)$	Database query engines, fast sequential range retrieval.

© Biraj